

Projekt zespołowy inżynierii oprogramowania

System zarządzania postępem prac zespołu
Bonnibel

Pavel Kapitsa, Margarita Kitaeva, Valadar Harelikau,
Sofiia Knyshoyid, Jan Jędra, Jarosław Klima,
Kacper Gałek, Kinga Kowal, Maciej Jamroży,
Mikołaj Kołek, Paweł Knot, Robert Jacak, Dawid Węcirz

Spis treści

Spis treści.....	2
Opis Projektu.....	4
Cel projektu.....	4
Główny scenariusz pracy.....	4
Problem, który rozwiązuje projekt.....	4
Role w systemie.....	5
Owner.....	5
Developer.....	5
Reviewer.....	5
Analiza wymagań funkcjonalnych.....	6
Zarządzanie kontem.....	6
Zarządzanie projektami.....	7
Zarządzanie uczestnikami projektu.....	7
Powiadomienia i komunikacja.....	8
Zarządzanie zadaniami i review.....	9
Zarządzanie zadaniami i analityka.....	10
Architektura modułowa.....	11
Diagrama sekwencji przetwarzanie zapytania.....	12
Moduły.....	13
Auth Module.....	13
System Strażników Tras (Protected Routes i Deklaratywny Routing).....	15
Dynamiczny Kontekstowy System Uprawnień (RBAC - Role-Based Access Control)...	15
Integration Module.....	18
Notification Module.....	19
Logic Modules.....	20
Project Service.....	21
Membership Service.....	22
System sprawdzania uprawnień.....	23
User Service.....	32
Cel i zakres odpowiedzialności.....	32
Struktura danych (modele domenowe).....	32
Specyfikacja metod serwisu.....	32
Task Service.....	34
Cel i zakres odpowiedzialności.....	34
Struktura danych (modele domenowe).....	34
Specyfikacja metod serwisu.....	34
Przykład implementacji w komponencie React.....	36
Docs Service.....	39

Cel i zakres odpowiedzialności.....	39
Struktura danych.....	39
Zasady walidacji i ograniczenia.....	39
Specyfikacja metod serwisu.....	39
Endpointy API.....	40
PR Service.....	43
Cel i zakres odpowiedzialności.....	43
Struktura danych.....	43
Zasady walidacji i ograniczenia.....	43
Specyfikacja metod serwisu.....	43
Endpointy API.....	44
Chat service.....	50
Cel i zakres odpowiedzialności.....	50
Struktura danych.....	50
Specyfikacja metod serwisu.....	50
Endpointy API.....	51
Komunikacja websocket.....	52
Analytics Service.....	53
Cel i zakres odpowiedzialności.....	53
Struktura danych.....	53
Specyfikacja metod serwisu.....	53
Endpointy API.....	54
Database structure.....	55
Project.....	57
ProjectMember.....	57
Auth.....	57
ProjectIntegration.....	57
Task.....	58
TaskSubscription.....	58
WebhookSecret.....	58
ChatMessage.....	59
PullRequest.....	59
TaskHistory.....	59
Notification.....	60

Opis Projektu

Cel projektu

Celem projektu jest stworzenie systemu, który integruje zadania, commity, dokumentację i komunikację zespołu w jednym miejscu. System pozwala śledzić postęp prac, ułatwia współpracę i automatyzuje procesy związane z zarządzaniem kodem i dokumentacją. Bonnibel działa jak „przewodnik zespołu”, nadzorując i kierując pracą członków zespołu w sposób przejrzysty i uporządkowany.

Główne funkcjonalności

- **Zarządzanie użytkownikami:** role (developer, reviewer, team lead), statusy (dostępny, zajęty, dostępny do zadań), subskrypcja na wybrane zadania.
- **Integracja z Jira:** automatyczne pojawianie się zadań, historia zadania, możliwość oznaczania punktów jako wykonane, aktualizacja statusów.
- **Integracja z Git:** automatyczne tworzenie gałęzi, śledzenie commitów, tworzenie pull requestów, powiadomienia dla reviewerów, automatyczne zamykanie gałęzi po zatwierdzeniu PR.
- **Dokumentacja:** możliwość dołączania dokumentacji do zadania i pull requestu.
- **Komunikacja:** chat przy każdym zadaniu, powiadomienia o nowych commitach, PR i zmianach statusów.
- **Analiza postępów:** statystyki pracy zespołu, historia zamkniętych zadań, kto czym się zajmuje i ile zadań zostało ukończonych.

Główny scenariusz pracy

1. Użytkownik otrzymuje zadanie z Jira.
2. System automatycznie tworzy gałąź w Git o nazwie użytkownika i numerze zadania.
3. Użytkownik wykonuje pracę i commit'uje zmiany.
4. Historia zadania aktualizuje się automatycznie z powiązaniem commitów.
5. Po zakończeniu pracy użytkownik dołącza dokumentację.
6. System tworzy pull request i powiadamia reviewera.
7. Po zatwierdzeniu PR gałąź zostaje zamknięta, a status zadania w Jira aktualizowany.
8. Użytkownicy mogą subskrybować zadania i przeglądać historię projektu.

Problem, który rozwiązuje projekt

Wiele zespołów programistycznych boryka się z chaosem informacyjnym: trudno śledzić postęp zadań, powiązania między kodem a dokumentacją oraz faktyczny status pracy członków zespołu. Bonnibel rozwiązuje te problemy, integrując Jira, Git i dokumentację w jednym miejscu, automatyzując procesy i zapewniając przejrzysty podgląd postępów dla całego zespołu.

Role w systemie

Owner

Użytkownik, który stworzył projekt.

Uprawnienia:

- Dodawanie i usuwanie uczestników projektu
- Zmiana ról użytkowników
- Tworzenie zadań
- Przypisywanie zadań użytkownikom
- Przeglądanie analityki projektu
- Zarządzanie projektem (edycja, usuwanie)
- Podłączanie integracji (Git, Jira, Docs)
- Otrzymywanie powiadomień o kluczowych zdarzeniach

Developer

Użytkownik odpowiedzialny za realizację zadań.

Uprawnienia:

- Przeglądanie swoich zadań
- Przeglądanie zadań projektu
- Aktualizacja statusu zadania
- Dodawanie dokumentacji do zadania
- Tworzenie requestu o zamknięcie zadania (PR)
- Subskrybowanie / odsubskrybowanie zadań
- Otrzymywanie powiadomień o zmianach w zadaniach
- Wysyłanie wiadomości w czacie zadania

Reviewer

Użytkownik odpowiedzialny za sprawdzanie zmian.

Uprawnienia:

- Otrzymywanie powiadomień o nowych Pull Requestach
- Akceptowanie zmian
- Odrzucanie zmian
- Przeglądanie dokumentacji zadania
- Uczestnictwo w procesie zamknięcia zadania
- Otrzymywanie powiadomień o statusie review

Analiza wymagań funkcjonalnych

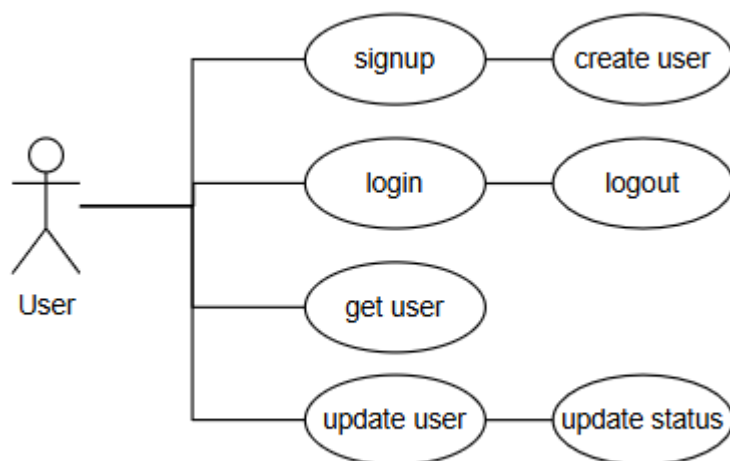
Zarządzanie kontem

Moduł zarządzania kontem umożliwia użytkownikowi wykonywanie podstawowych operacji związanych z obsługą konta w systemie. Obejmuje procesy rejestracji (signup), logowania (login) oraz wylogowania (logout).

Po rejestracji użytkownik zobowiązany jest do utworzenia profilu, co stanowi pierwszy krok po założeniu konta. Następnie może się zalogować i korzystać z funkcjonalności systemu.

Wylogowanie realizowane jest wyłącznie po stronie klienta poprzez usunięcie refresh tokena, bez dodatkowej operacji po stronie serwera.

Po zalogowaniu użytkownik ma dostęp do swojego profilu — może pobrać dane (get user) oraz je edytować (update user). W ramach edycji możliwa jest także zmiana statusu (update status), który określa dostępność użytkownika w systemie (np. dostępny, zajęty).



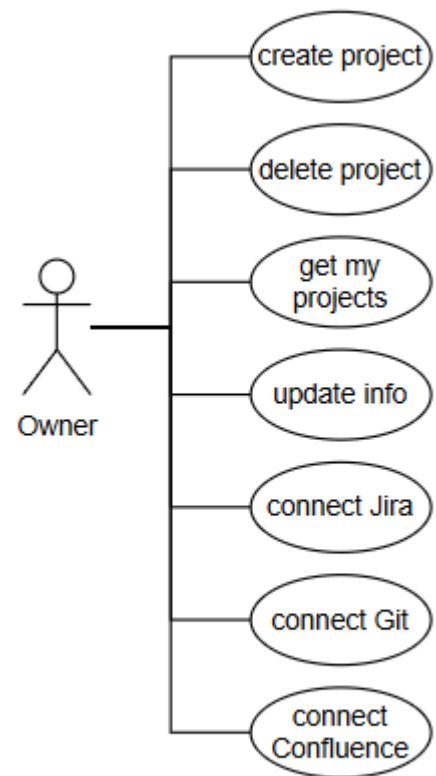
Zarządzanie projektami

Moduł zarządzania projektami umożliwia użytkownikowi z rolą Owner zarządzanie projektami w systemie. Obejmuje on operacje tworzenia (create project), usuwania (delete project), pobierania listy projektów użytkownika (get my projects) oraz aktualizacji informacji o projekcie (update info).

Po utworzeniu projektu użytkownik automatycznie otrzymuje rolę Owner, co daje mu pełną kontrolę nad projektem oraz jego zasobami.

Owner może zarządzać konfiguracją projektu, w tym integrować go z zewnętrznymi systemami. Dostępne są operacje podłączenia Jira (connect Jira), Git (connect Git) oraz systemu dokumentacji (connect Confluence), co umożliwia synchronizację zadań, kodu oraz dokumentacji.

Wszystkie operacje rozszerzają podstawową funkcję zarządzania projektem. Diagram przypadków użycia przedstawia zarówno główne działania związane z projektem, jak i dodatkowe funkcjonalności związane z integracjami.



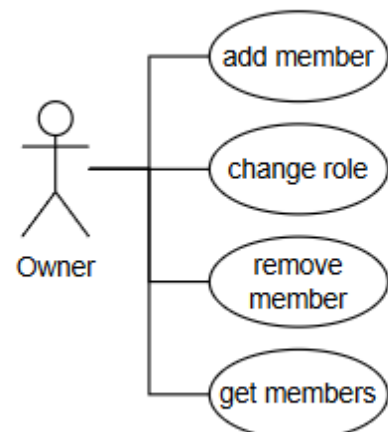
Zarządzanie uczestnikami projektu

Moduł zarządzania uczestnikami umożliwia użytkownikowi z rolą Owner kontrolowanie składu zespołu w projekcie. Obejmuje on operacje dodawania użytkowników do projektu (add member), usuwania ich (remove member) oraz pobierania listy uczestników (get members).

Podczas dodawania użytkownika do projektu przypisywana jest mu określona rola, która definiuje jego uprawnienia w systemie. Owner ma możliwość zmiany tej roli, co stanowi rozszerzenie operacji zarządzania członkami projektu.

Usunięcie użytkownika powoduje utratę dostępu do projektu i wszystkich powiązanych z nim zasobów. Funkcja pobierania listy uczestników pozwala na wgląd w aktualny skład zespołu wraz z przypisanymi rolami.

Diagram przypadków użycia przedstawia podstawowe operacje zarządzania członkami oraz ich rozszerzenia związane z kontrolą ról i dostępu.



Powiadomienia i komunikacja

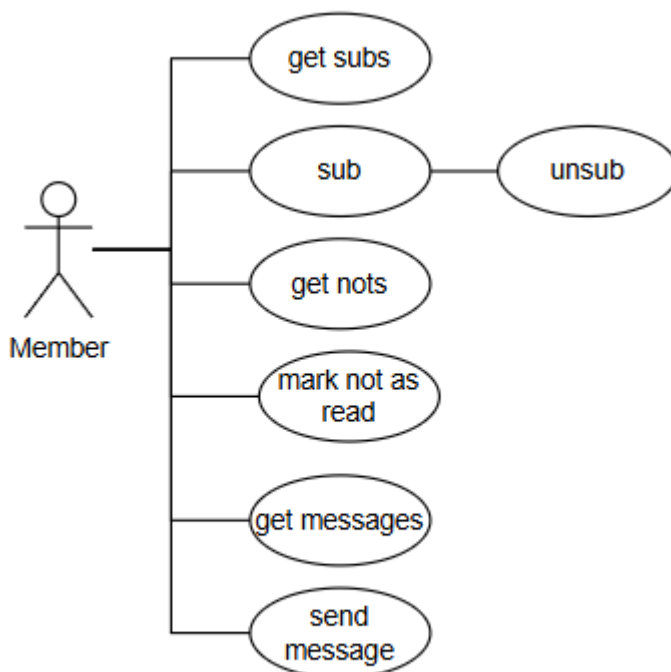
Moduł powiadomień i komunikacji umożliwia każdemu uczestnikowi projektu (Member) zarządzanie subskrypcjami, odbieranie powiadomień oraz komunikację w ramach zadań.

Użytkownik może subskrybować zadania (sub), aby otrzymywać informacje o zmianach z nimi związanych, oraz rezygnować z subskrypcji (unsub). Dodatkowo możliwe jest pobranie listy subskrypcji (get subs), co pozwala kontrolować obserwowane elementy.

System udostępnia również funkcje związane z powiadomieniami — użytkownik może pobrać swoje powiadomienia (get notifications) oraz oznaczyć je jako przeczytane (mark not as read). Powiadomienia informują o kluczowych zdarzeniach w projekcie, takich jak zmiana statusu zadania, nowe commity czy pull requesty.

W ramach komunikacji użytkownik może przeglądać wiadomości w czacie zadania (get messages) oraz wysyłać nowe wiadomości (send message). Każda wiadomość jest powiązana z konkretnym zadaniem, a jej wysłanie powoduje wysłanie powiadomień do subskrybujących użytkowników.

Diagram przypadków użycia przedstawia podstawowe operacje związane z subskrypcjami, powiadomieniami oraz komunikacją, które wspierają współpracę w projekcie.



Zarządzanie zadaniami i review

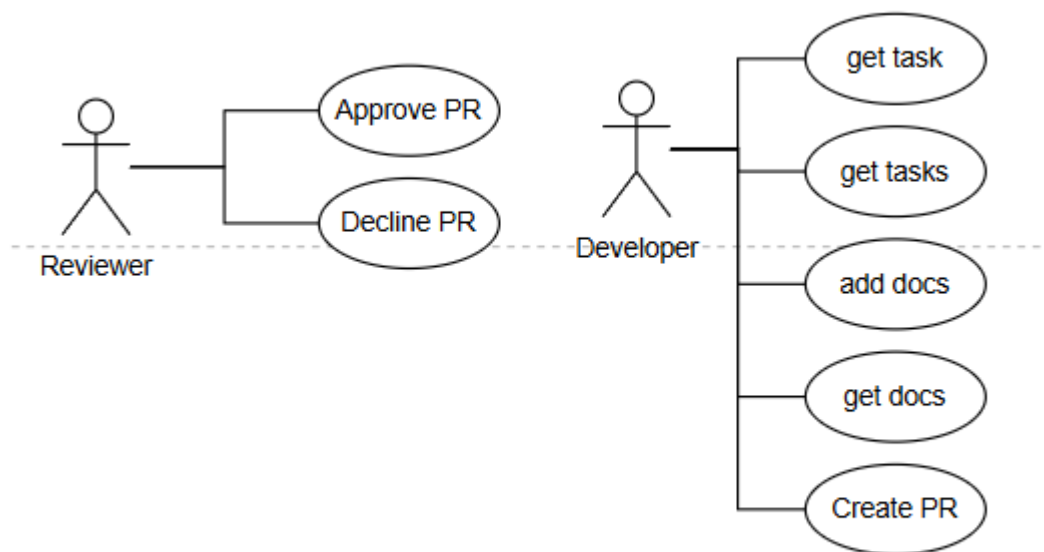
Moduł zarządzania zadaniami obejmuje operacje dostępne dla użytkowników z rolami Developer oraz Reviewer i odpowiada za realizację oraz weryfikację pracy nad zadaniami.

Developer ma możliwość przeglądania pojedynczego zadania (get task) oraz listy zadań (get tasks). W trakcie realizacji zadania może dodawać dokumentację (add docs) oraz ją przeglądać (get docs). Dokumentacja stanowi istotny element procesu i jest wymagana do zakończenia zadania.

Po zakończeniu pracy Developer może utworzyć pull request (create PR), który inicjuje proces weryfikacji zmian.

Reviewer odpowiada za ocenę zgłoszonych zmian — może zaakceptować pull request (approve PR) lub go odrzucić (decline PR). Decyzja reviewera wpływa na dalszy przebieg procesu — w przypadku akceptacji możliwe jest zamknięcie zadania i synchronizacja z systemami zewnętrznymi, natomiast odrzucenie wymaga wprowadzenia poprawek przez developera.

Diagram przypadków użycia przedstawia podział odpowiedzialności między Developerem i Reviewerem oraz główne operacje związane z realizacją i weryfikacją zadań.

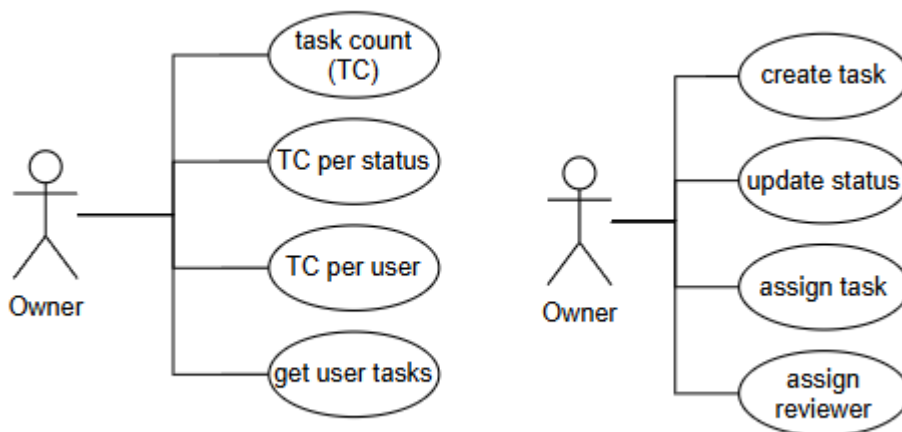


Zarządzanie zadaniami i analityka

Moduł zarządzania zadaniami umożliwia użytkownikowi z rolą Owner tworzenie i kontrolowanie przebiegu realizacji zadań w projekcie. Obejmuje on operacje tworzenia zadania (create task), przypisywania go do użytkownika (assign task) oraz przypisywania reviewera (assign reviewer). Owner może również aktualizować status zadania (update status), co pozwala na śledzenie postępu prac.

Równolegle moduł analityki dostarcza informacji o stanie projektu. Owner może pobrać liczbę zadań w projekcie (task count), analizować ich rozkład według statusów (task count per status), a także według użytkowników (task count per user). Dodatkowo możliwe jest pobranie listy zadań przypisanych do konkretnego użytkownika (get user tasks).

Funkcjonalności analityczne stanowią rozszerzenie podstawowego zarządzania zadaniami i umożliwiają bieżącą ocenę postępu prac zespołu. Diagram przypadków użycia przedstawia zarówno operacje zarządzania zadaniami, jak i dostępne mechanizmy analizy danych.



Architektura modułowa

Architektura systemu Bonnibel oparta jest na klasycznym podejściu warstwowym (tzw. „layered architecture”), w którym poszczególne moduły są odpowiedzialne za określone aspekty działania systemu i komunikują się między sobą w sposób uporządkowany.

Na najwyższym poziomie znajdują się warstwy wejściowe, które odpowiadają za przyjmowanie żądań z zewnątrz. Są to **REST Controllers**, obsługujące zapytania od użytkowników, oraz **Hook Handlers**, które odbierają zdarzenia z zewnętrznych systemów, takich jak Git czy Jira.

Każda z tych ścieżek posiada własny mechanizm uwierzytelniania. Dla żądań użytkownika wykorzystywany jest **Auth Module**, który odpowiada za weryfikację tokenów oraz kontrolę dostępu. Dla zdarzeń zewnętrznych stosowany jest **Hook Auth Module**, który weryfikuje autentyczność webhooków na podstawie podpisów i sekretów integracji.

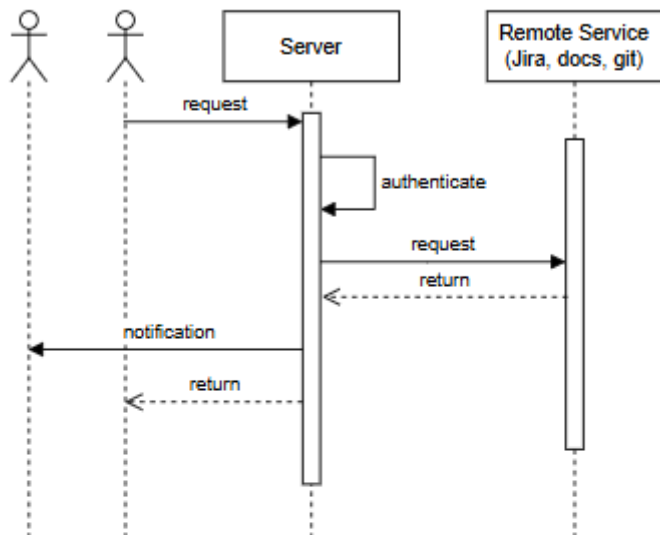
Centralną część systemu stanowią **Business Modules**, które implementują właściwą logikę biznesową. To tutaj realizowane są wszystkie przypadki użycia, takie jak zarządzanie projektami, zadaniami, użytkownikami czy dokumentacją. Moduły te nie zależą bezpośrednio od warstwy transportowej ani od szczegółów integracji, co zapewnia ich niezależność i czytelność.

Na najniższym poziomie znajdują się moduły odpowiedzialne za komunikację z systemami zewnętrznymi oraz użytkownikami. **Integration Module** obsługuje połączenia z usługami takimi jak Git, Jira czy systemy dokumentacji, realizując zapytania i synchronizację danych. Z kolei **Notification Module** odpowiada za dostarczanie powiadomień do użytkowników, w tym przez WebSocket, oraz za zarządzanie ich stanem (np. odczytane/nieodczytane).

Hook handlers	Rest Controllers
Hook Auth Module	Auth Module
Business Modules	
Integration Module	Notification Module

Diagrama sekwencji przetwarzanie zapytania

Przetwarzanie zapytania od usera



Po otrzymaniu zapytania system w pierwszej kolejności przeprowadza uwierzytelnienie użytkownika oraz weryfikację jego uprawnień.

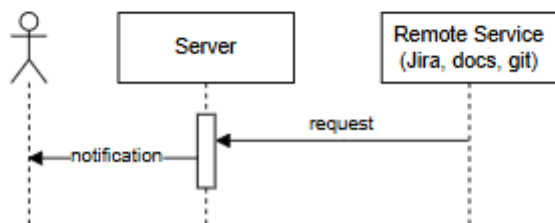
Następnie wykonywane są odpowiednie operacje w systemie Bonnibel, takie jak odczyt lub modyfikacja danych, zapis zmian w bazie oraz aktualizacja stanu powiązanych encji.

Po przetworzeniu zapytania system przekazuje informacje o zmianach:

- do użytkowników — w postaci powiadomień,
- do zewnętrznych serwisów — poprzez integracje realizowane za pomocą komunikacji REST.

Po otrzymaniu żądania z zewnętrznego serwisu system najpierw weryfikuje jego autentyczność na podstawie podpisu (webhook authentication).

Przetwarzanie hooka od servisa



Następnie wykonywane są odpowiednie operacje w systemie Bonnibel, takie jak aktualizacja danych, zapis zmian w bazie oraz synchronizacja stanu encji z systemem wewnętrznym.

Po przetworzeniu zdarzenia system przekazuje informacje o zmianach do użytkowników w postaci powiadomień,

Żądanie od serwisu nie zwraca danych biznesowych — jedynie potwierdza poprawne przetworzenie.

Moduły

Auth Module

Moduł Auth odpowiada za bezpieczne uwierzytelnianie użytkowników oraz ścisłą kontrolę dostępu do zasobów systemu. Oparty jest na standardzie JWT (JSON Web Tokens) oraz natywnym dla FastAPI mechanizmie wstrzykiwania zależności (Dependency Injection). Architektura modułu dzieli się na dwa główne procesy: uwierzytelnianie (kto wykonuje żądanie) oraz autoryzację (co może zrobić).

1. Uwierzytelnianie (AuthService i zarządzanie sesją) Procesy związane z logowaniem i rejestracją realizowane są w warstwie usług (AuthService).

Przetwarzanie haseł: System nie przechowuje haseł w postaci jawnej. Za ich zabezpieczenie odpowiada wewnętrzny moduł bezpieczeństwa wykorzystujący algorytm bcrypt. Każde hasło przed zapisem do bazy jest hashowane z użyciem unikalnej soli.

Strategia Tokenów (Access i Refresh): Po pomyślnym uwierzytelnieniu (/api/auth/login), użytkownik otrzymuje dwa tokeny generowane przez dedykowany serwis:

- Access Token: Krótko żyjący (np. 30 minut), bezstanowy token JWT. Przechowuje wyłącznie identyfikator użytkownika (user_id). Jest on dołączany do każdego żądania HTTP w nagłówku Authorization: Bearer.
 - Refresh Token: Długo żyjący (np. 7 dni), kryptograficznie bezpieczny ciąg znaków, który jest fizycznie zapisywany w bazie danych (tabela refresh_tokens). Pozwala on na odnowienie wygasłego Access Tokena, a także daje możliwość natychmiastowego zablokowania sesji (flaga is_revoked) z poziomu serwera.
2. Autoryzacja i Kontrola Dostępu (AuthGuard i RoleChecker) Weryfikacja uprawnień do konkretnych endpointów odbywa się w locie (przy każdym żądaniu HTTP) i jest całkowicie oddzielona od procesu weryfikacji tożsamości. Realizuje to klasa RoleChecker pełniąca funkcję Guarda.

Proces weryfikacji żądania przebiega w następujących krokach:

1. Identyfikacja (get_current_user): Guard dekoduje nadesłany Access Token, weryfikuje jego poprawność kryptograficzną i wyciąga identyfikator użytkownika.
2. Rozpoznanie kontekstu: Guard automatycznie ekstrahuje identyfikator projektu (project_id) bezpośrednio z parametrów ścieżki żądania (URL).
3. Weryfikacja bazy danych: Guard wykonuje szybkie zapytanie do tabeli project_members, sprawdzając powiązanie między zidentyfikowanym użytkownikiem a danym projektem.
4. Walidacja roli: Pobrana z bazy rola (np. OWNER, DEVELOPER, REVIEWER) jest zestawiana z uprawnieniami wymaganymi do wykonania danej operacji (endpointu).

Korzyści z zastosowanego podziału:

Bezpieczeństwo i wydajność: Role nie są zapisywane w Access Tokenie. Dzięki odpytywaniu bazy danych (tabeli project_members) o uprawnienia przy każdym chronionym

żądaniu, zmiana roli użytkownika lub jego usunięcie z projektu skutkuje natychmiastową utratą dostępu do zasobów (brak problemu opóźnień związanych z cyklem życia JWT).

Czystość kodu: Logika kontroli dostępu jest wstrzykiwana bezpośrednio do sygnatur endpointów w routerach (np. `membership = Depends(require_owner)`). Kod kontrolerów pozostaje wolny od instrukcji warunkowych związanych z uprawnieniami, skupiając się wyłącznie na logice biznesowej.

Mechanizm działania krok po kroku opisany od frontendu aż do backendu:

1. **Rejestracja (Signup):** Użytkownik przesyła dane (Email, Name, Surname, Password) przez formularz walidowany po stronie klienta. Frontend wykonuje żądanie POST. Po pomyślnym utworzeniu rekordu w bazie, użytkownik jest przekierowywany do widoku logowania.
2. Użytkownik wprowadza dane logowania (adres e-mail, hasło) w formularzu strony `LoginPage`. Interfejs kontroluje poprawność wprowadzonych ciągów znaków za pomocą biblioteki `react-hook-form`.
3. Po wywołaniu zdarzenia przesłania formularza, dane są przekazywane do metody `authService.login()`, która wysyła żądanie strukturą JSON metodą POST na endpoint `/auth/login`.
4. Backend sprawdza poświadczenia, generuje i zwraca obiekt `AuthResponse` zawierający parametry `access_token`, `refresh_token` oraz unikalny identyfikator użytkownika `user_id`.
5. Funkcja zwrotna przechwytyje odpowiedź, wywołuje metodę `login` z globalnego kontenera `useAuthStore`, co skutkuje trwałym zapisaniem tokenów w przestrzeni `localStorage` przeglądarki i zmianą flagi reaktywnej `isAuthenticated` na wartość `true`.
6. Aplikacja wykonuje programowe przekierowanie użytkownika na widok zabezpieczony (np. profil lub domyślną listę projektów).
7. **Wylogowanie (Logout):** Proces ten realizowany jest całkowicie po stronie klienta. Wywołanie akcji wylogowania powoduje natychmiastowe wyczyszczenie kluczy autoryzacyjnych z pamięci podręcznej przeglądarki, unieważnienie lokalnego stanu i twarde przekierowanie użytkownika na ekran `/login`.

Wykorzystywane Kontrakty i Endpointy API:

- **POST /api/auth/signup** – Rejestracja nowego konta.
 - *Parametry wejściowe:* `{ "email": "string", "password": "string", "name": "string", "surname": "string" }`
 - *Struktura odpowiedzi:* `{ "message": "string", "user_id": "string" }`
 -
- **POST /api/auth/login** – Logowanie i inicjalizacja sesji.
 - *Parametry wejściowe:* `{ "email": "string", "password": "string" }`

- *Struktura odpowiedzi:* { "access_token": "string", "refresh_token": "string", "user_id": "string" }
- **POST /auth/refresh?refresh_token={token}** – Żądanie rotacji wygasłego klucza sesyjnego (wywoływane automatycznie przez warstwę sieciową).

System Strażników Tras (Protected Routes i Deklaratywny Routing)

Bezpieczeństwo integralności nawigacji systemu Bonnibel jest realizowane na poziomie definicji drzewa routingu aplikacji za pomocą komponentu `createBrowserRouter`. Trasy systemu zostały jawnie podzielone na strefy publiczne (Dostępne bez uwierzytelnienia, np. formularze logowania i rejestracji ustrukturyzowane wewnątrz kontenera wizualnego `AuthLayout`) oraz strefy chronione (Zabezpieczone komponentem `ProtectedRoute`).

Kod źródłowy: Komponent Strażnika Ścieżki (`ProtectedRoute.tsx`)

```
import { Navigate, Outlet } from 'react-router-dom';
import { useAuthStore } from '../modules/auth/store/authStore';

export const ProtectedRoute = () => {
  const isAuthenticated = useAuthStore((state) =>
state.isAuthenticated);

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  return <Outlet />;
};
```

Próba bezpośredniego wpisania w pasek przeglądarki dowolnego chronionego adresu (np. `/projects/1/tasks`) skutkuje przechwyceniem żądania przez komponent `ProtectedRoute`. Jeśli flaga `isAuthenticated` pobrana ze stanu globalnego ma wartość `false`, aplikacja bezwarunkowo przerywa cykl życia widoku docelowego i przekierowuje użytkownika na ścieżkę publiczną `/login`.

Dynamiczny Kontekstowy System Uprawnień (RBAC - Role-Based Access Control)

System Bonnibel wdraża granularne zarządzanie uprawnieniami kontekstowymi powiązanymi z rolą użytkownika w danym, konkretnym projekcie. Wartości uprawnień definiuje typ unii `ProjectRole = 'OWNER' | 'DEVELOPER' | 'REVIEWER'`.

Zamiast pobierania ról przy logowaniu dla całej aplikacji, system wykonuje zapytanie o uprawnienia dynamicznie, w locie, przy wejściu do dowolnego zasobu projektowego za pomocą metody `projectService.listMembers(projectId)`. Zabezpiecza to aplikację przed niespójnością danych w przypadku zmiany roli użytkownika w trakcie trwania jego sesji.

Tabela Uprawnień Biznesowych w Widoku Szczegółów Zadania (`TaskDetailsPage`)

Uprawnienie funkcjonalne	Rola: OWNER	Rola: DEVELOPER	Rola: REVIEWER
Dostęp do konfiguracji Integracji (<code>/integrations</code>)	TAK	NIE	NIE
Możliwość utworzenia zadania	TAK	TAK	NIE
Zmiana statusu zadania na: <code>TODO</code> , <code>IN_PROGRESS</code>	TAK	TAK	NIE
Przekazanie zadania do statusu: <code>IN_REVIEW</code>	TAK	TAK	TAK
Zamknięcie zadania i akceptacja: <code>DONE</code> , <code>CLOSED</code>	TAK	NIE	TAK
Edycja i zmiana przypisanej osoby (Assignee)	TAK	TAK	NIE
Edycja i zmiana recenzenta zadania (Reviewer)	TAK	NIE	TAK

```
type ProjectMemberApi = {  
  
  project_id?: number  
  
  projectId?: number
```



```
user_id?: string

userId?: string

role: ProjectRole
```

```
listMembers: async (projectId: number): Promise<ProjectMember[]> => {
  const members = await
apiFetch<ProjectMemberApi[]>(`/project/${projectId}/members`)
  return members.map(mapMember)
},
```

Wykorzystywane Kontrakty i Endpointy API Modułu Projektów:

- GET /project/{projectId}/members – Pobranie listy uprawnionych uczestników projektu.
 - Odpowiedź (JSON): [{ "project_id": 1, "user_id": "usr_abc", "role": "DEVELOPER" }]
- PATCH /project/{projectId}/members/{userId}/role – Modyfikacja roli użytkownika w strukturze projektowej (Uprawnienie zastrzeżone dla roli OWNER).
 - Parametry wejściowe: { "role": "REVIEWER" }

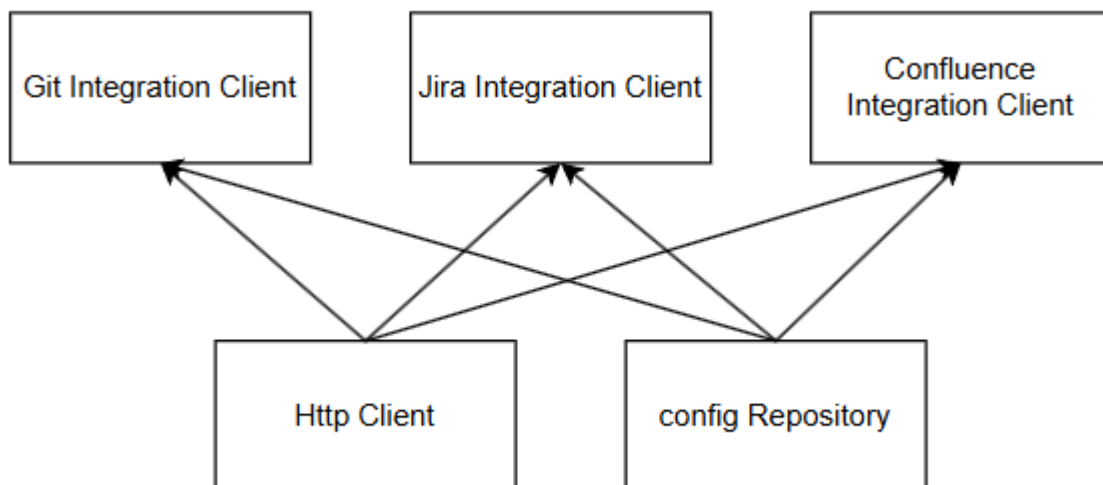
Integration Module

Moduł integracji odpowiada za komunikację z zewnętrznymi systemami, takimi jak Git, Jira oraz system dokumentacji (np. Confluence). Jego celem jest umożliwienie synchronizacji danych oraz wykonywania operacji w tych systemach bezpośrednio z poziomu Bonnibel.

Główne komponenty modułu to wyspecjalizowane klienty integracyjne: **Git Integration Client**, **Jira Integration Client** oraz **Confluence Integration Client**. Każdy z nich implementuje operacje charakterystyczne dla danego systemu, takie jak tworzenie pull requestów, zarządzanie zadaniami w Jira czy obsługa dokumentacji.

Wszystkie klienty korzystają ze wspólnego **Http Client**, który odpowiada za wykonywanie zapytań HTTP, obsługę autoryzacji oraz błędów komunikacji. Dzięki temu logika komunikacji sieciowej jest ujednolicona i nie jest duplikowana w poszczególnych integracjach.

Dodatkowo klienty wykorzystują **Config Repository**, który dostarcza dane konfiguracyjne niezbędne do komunikacji z zewnętrznymi usługami, takie jak tokeny dostępu czy identyfikatory integracji. Repozytorium to stanowi warstwę dostępu do bazy danych i ukrywa szczegóły przechowywania konfiguracji.



Notification Module

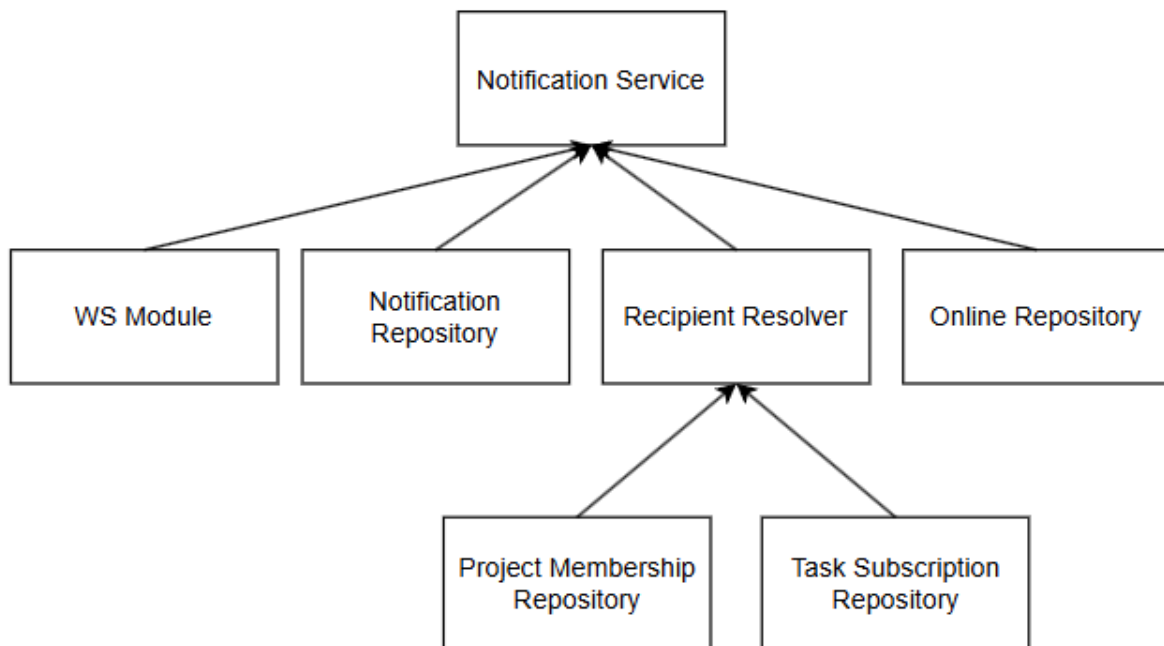
Moduł powiadomień odpowiada za dostarczanie informacji o zdarzeniach w systemie do odpowiednich użytkowników. Obsługuje zarówno wybór odbiorców, jak i samo wysyłanie oraz przechowywanie powiadomień.

Centralnym elementem modułu jest **Notification Service**, który orkiestruje cały proces — od przyjęcia zdarzenia, przez wyznaczenie odbiorców, aż po wysłanie powiadomień.

Do komunikacji w czasie rzeczywistym wykorzystywany jest **WS Module**, który odpowiada za wysyłanie powiadomień do aktualnie połączonych użytkowników. Informacja o tym, czy użytkownik jest online, przechowywana jest w **Online Repository**.

Powiadomienia są zapisywane w bazie danych za pomocą **Notification Repository**, co pozwala użytkownikowi na ich późniejsze przeglądanie oraz oznaczanie jako przeczytane.

Dobór odbiorców realizowany jest przez **Recipient Resolver**, który na podstawie typu zdarzenia określa, do kogo powinno zostać wysłane powiadomienie. W tym celu korzysta on z **Project Membership Repository** (np. do znalezienia ownera, reviewera lub uczestników projektu) oraz **Task Subscription Repository** (do określenia użytkowników subskrybujących dane zadanie).

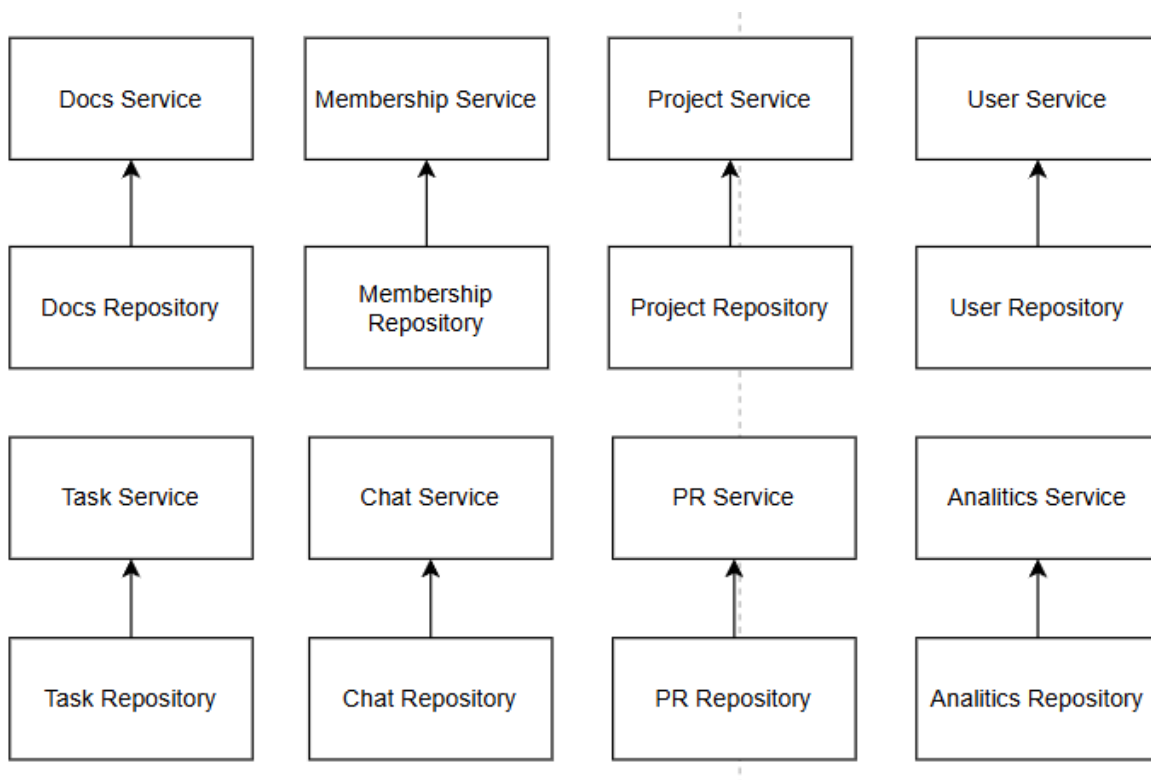


Logic Modules

Warstwa logiki biznesowej składa się z wyspecjalizowanych modułów, z których każdy odpowiada za konkretny obszar funkcjonalny systemu:

- **Project Service** — odpowiada za zarządzanie projektami, w tym ich tworzenie, aktualizację, usuwanie oraz konfigurację integracji.
- **Membership Service** — zarządza uczestnikami projektu, ich rolami oraz dostępem do zasobów.
- **User Service** — obsługuje dane użytkownika, umożliwiając ich pobieranie, edycję oraz aktualizację statusu dostępności.
- **Task Service** — realizuje logikę związaną z zadaniami, w tym ich tworzenie, przypisywanie, zmianę statusów oraz przepływ pracy.
- **Docs Service** — odpowiada za dodawanie i pobieranie dokumentacji powiązanej z zadaniami oraz integrację z systemami dokumentacji.
- **PR Service** — obsługuje proces pull requestów, w tym ich tworzenie oraz zarządzanie cyklem review.
- **Chat Service** — umożliwia komunikację w ramach zadań poprzez wysyłanie i odczytywanie wiadomości.
- **Analytics Service** — dostarcza dane statystyczne dotyczące projektu, takie jak liczba zadań, ich statusy oraz aktywność użytkowników.

Każdy z modułów zawiera własną logikę biznesową oraz korzysta z repozytoriów dopasowanych do swoich potrzeb, bez wydzielania osobnej globalnej warstwy dostępu do bazy danych.



Project Service

Cel i zakres odpowiedzialności

ProjectService jest centralnym serwisem odpowiedzialnym za zarządzanie cyklami życia projektów w systemie Bonnibel. Odpowiada za tworzenie, odczyt, aktualizację oraz usuwanie projektów, a także za zarządzanie integracjami zewnętrznymi.

Serwis ściśle współpracuje z MembershipService w celu automatycznego nadawania roli **OWNER** twórcy projektu oraz egzekwowania reguł dostępu opartych na rolach.

Struktura danych

Główne encje:

- **Project** - podstawowe informacje o projekcie
- **ProjectMember** - relacja wielu-do-wielu z rolami
- **ProjectIntegration** - konfiguracja integracji (Git, Jira, Confluence)
- **WebhookSecret** - sekrety webhooków

Specyfikacja metod serwisu

1. **create_project(owner_id, name, description)**
 - Tworzy nowy projekt.
 - Automatycznie dodaje twórcę jako **OWNER** w tabeli project_members.
 - Zwraca utworzony projekt.
2. **get_user_projects(user_id)**
 - Zwraca listę wszystkich projektów, w których użytkownik jest członkiem (dowolna rola).
3. **get_project(user_id, project_id)**
 - Pobiera szczegóły projektu.
 - Wymaga co najmniej roli **DEVELOPER** (lub wyższej).
4. **update_project(user_id, project_id, name, description)**
 - Aktualizuje nazwę i/lub opis projektu.
 - **Tylko OWNER** projektu może wykonać tę operację.
5. **delete_project(user_id, project_id)**
 - Usuwa projekt wraz ze wszystkimi powiązanymi danymi (członkowie, integracje, webhook secrets).
 - **Tylko OWNER** może usunąć projekt.
6. **Operacje integracji** (connect_integration, disconnect_integration, list_integrations)
 - Zarządzanie integracjami projektu.
 - Wszystkie operacje wymagają roli **OWNER** (oprócz odczytu - wystarczy **DEVELOPER**).

Endpointy API

- POST /api/project/ - utworzenie projektu

- GET /api/project/my - lista projektów użytkownika
- GET /api/project/{project_id} - szczegóły projektu
- PUT /api/project/{project_id} - edycja projektu
- DELETE /api/project/{project_id} - usunięcie projektu
- POST /api/project/{project_id}/integrations - podłączenie integracji
- GET /api/project/{project_id}/integrations - lista integracji
- DELETE /api/project/{project_id}/integrations/{provider} - odłączenie integracji

Reguły biznesowe

- Twórca projektu automatycznie staje się jego **OWNERem**.
- Tylko **OWNER** może edytować lub usuwać projekt.
- Tylko **OWNER** może zarządzać integracjami.

Membership Service

Cel i zakres odpowiedzialności

MembershipService zarządza członkostwem w projektach oraz rolami użytkowników. Zapewnia granularną kontrolę dostępu na poziomie pojedynczego projektu.

Struktura danych

- **ProjectMember** (klucz złożony: project_id + user_id)
 - role: OWNER | DEVELOPER | REVIEWER

Specyfikacja metod serwisu

1. **add_member(actor_id, project_id, user_id, role)**
 - Dodaje użytkownika do projektu.
 - **Tylko OWNER** może dodać członka.
 - Nie pozwala dodać użytkownika, który już jest w projekcie.
2. **remove_member(actor_id, project_id, user_id)**
 - Usuwa użytkownika z projektu.
 - **Tylko OWNER** może usuwać.
 - **Zabronione** jest usunięcie użytkownika z rolą OWNER.
3. **change_member_role(actor_id, project_id, user_id, new_role)**
 - Zmienia rolę użytkownika w projekcie.
 - **Tylko OWNER** może zmieniać role.
 - Nie pozwala zmienić roli OWNER na inną.
4. **get_project_members(actor_id, project_id)**
 - Zwraca listę wszystkich członków projektu.

- Wymaga co najmniej roli **DEVELOPER**.

Endpointy API

- POST /api/project/{project_id}/members
- GET /api/project/{project_id}/members
- PATCH /api/project/{project_id}/members/{user_id}/role
- DELETE /api/project/{project_id}/members/{user_id}

Reguły biznesowe

- Rola jest **kontekstowa** (per-projekt) - jeden użytkownik może być OWNERem w jednym projekcie, a DEVELOPERem w innym.
- Nikt nie może usunąć OWNERa projektu.
- OWNER nie może zmienić swojej roli na niższą.
- Wszystkie operacje członkostwa są chronione przez `_require_project_role()`.

System sprawdzania uprawnień

W module logic zaimplementowano funkcję pomocniczą:

```
_require_project_role(db, user_id, project_id, required_role=ProjectRole.OWNER)
```

która:

- Sprawdza czy użytkownik należy do projektu.
- Weryfikuje wymaganą rolę (dla OWNER - dokładne dopasowanie, dla DEVELOPER/REVIEWER - wystarczająca przynależność).
- Rzuci HTTP 403 z odpowiednim komunikatem (NOT_A_PROJECT_MEMBER, ONLY_OWNER_ALLOWED).

Project & Membership –

Zakres odpowiedzialności frontendu

Frontend modułu Project & Membership udostępnia użytkownikowi ekrany do przejścia od listy projektów do szczegółów wybranego projektu, a następnie do zarządzania członkami, rolami i integracjami. Logika biznesowa pozostaje po stronie backendu; frontend odpowiada za wywołania API, prezentację danych, obsługę stanów ładowania i komunikaty dla użytkownika.

- Pobranie i prezentacja listy projektów użytkownika.
- Wyświetlenie szczegółów projektu oraz nawigacji do modułów: zadania, członkowie, analityka i integracje.
- Pobranie listy członków projektu wraz z rolami OWNER, DEVELOPER i REVIEWER.

- Dodanie członka do projektu przez podanie user_id i roli.
- Zmiana roli istniejącego członka projektu.
- Usunięcie członka projektu z blokadą dla właściciela projektu.
- Podłączenie, wyświetlenie i odłączenie integracji GitHub, Jira lub Confluence.

Routing i widoki React

Trasy modułu są częścią chronionej części aplikacji. Dostęp do nich wymaga zalogowania, ponieważ są zagnieżdżone pod ProtectedRoute i AppLayout.

Ścieżka	Komponent	Odpowiedzialność widoku
/projects	ProjectsListPage	Lista projektów dostępnych dla użytkownika; karty projektów; komunikaty ładowania, błędu i pustej listy.
/projects/:projectId	ProjectDetailsPage	Szczegóły projektu oraz linki do sekcji: Zadania, Członkowie, Analityka, Integracje.
/projects/:projectId/members	MembersPage	Formularz dodawania członka, tabela członków, zmiana roli i usuwanie członka.
/projects/:projectId/integrations	ProjectIntegrationsPage	Lista integracji, podłączenie providera, external_id, token dostępu i odłączenie integracji.

Warstwa serwisów frontendowych

Komunikacja z backendem jest wydzielona do plików src/services. Komponenty nie składają ręcznie endpointów, tylko korzystają z metod domenowych. W Project & Membership najważniejsze są projectService oraz integrationService.

Serwis	Metody	Zastosowanie
projectService	list(), get(projectId)	Pobieranie listy projektów i danych pojedynczego projektu.
projectService	listMembers(projectId)	Pobranie uczestników projektu. Backend zwraca

		snake_case, frontend mapuje na camelCase.
projectService	addMember(projectId, data)	Dodanie użytkownika do projektu z rolą OWNER, DEVELOPER lub REVIEWER.
projectService	changeMemberRole(projectId, userId, role)	Zmiana roli członka projektu.
projectService	removeMember(projectId, userId)	Usunięcie członka projektu.
integrationService	list(), connect(), disconnect()	Lista integracji, połączenie i odłączenie GitHub/Jira/Confluence.

Modele danych TypeScript używane przez frontend

```

type ProjectRole = 'OWNER' | 'DEVELOPER' | 'REVIEWER'

type ProjectSummary = {
  projectId: number
  name: string
  description?: string
  ownerId?: string
}

type ProjectMember = {
  projectId: number
  userId: string
  role: ProjectRole
}

type ProjectIntegration = {
  integration_id: number
  project_id: number
  provider: 'GITHUB' | 'JIRA' | 'CONFLUENCE'
  external_id: string
  is_active: boolean
}

```

Przepływ użytkownika

1. Użytkownik loguje się i przechodzi do /projects.
2. ProjectsListPage pobiera projekty i renderuje kafelki. Kliknięcie projektu prowadzi do /projects/{projectId}.
3. ProjectDetailsPage pokazuje nazwę, opis, identyfikator właściciela oraz skróty do podmodułów.

4. MembersPage pobiera członków projektu. Owner może dodać osobę, zmienić jej rolę albo usunąć ją z projektu.
5. ProjectIntegrationsPage pobiera integracje. Owner może podłączyć GitHub, Jira lub Confluence przez external_id i token dostępu.

Kontrakty API – Project

Backendowy router projektu działa pod prefiksem /api/project. Jeżeli apiFetch dodaje prefiks /api automatycznie, frontend może wywoływać ścieżki bez pierwszego członu /api.

Utworzenie projektu

- Endpoint: POST /api/project/
- Request: { "name": "Bonnibel", "description": "Opis projektu" }
- Response: { "project_id": 1, "owner_id": "usr_1", "name": "Bonnibel", "description": "Opis projektu" }
- Błędy: 422 – błędne dane wejściowe

Lista moich projektów

- Endpoint: GET /api/project/my
- Request: brak
- Response: [{ "project_id": 1, "name": "Bonnibel", "description": "Opis", "owner_id": "usr_1" }]
- Błędy: 401 – brak autoryzacji

Szczegóły projektu

- Endpoint: GET /api/project/{project_id}
- Request: brak
- Response: { "project_id": 1, "owner_id": "usr_1", "name": "Bonnibel", "description": "Opis" }
- Błędy: 403 NOT_A_PROJECT_MEMBER, 404 PROJECT_NOT_FOUND

Edycja projektu

- Endpoint: PUT /api/project/{project_id}
- Request: { "name": "Nowa nazwa", "description": "Nowy opis" }
- Response: { "project_id": 1, "owner_id": "usr_1", "name": "Nowa nazwa", "description": "Nowy opis" }

- Błędy: 403 ONLY_OWNER_ALLOWED, 404 PROJECT_NOT_FOUND, 422

Usunięcie projektu

- Endpoint: DELETE /api/project/{project_id}
- Request: brak
- Response: 204 No Content
- Błędy: 403 ONLY_OWNER_ALLOWED

Kontrakty API – Membership

Dodanie członka

- Endpoint: POST /api/project/{project_id}/members
- Request: { "user_id": "usr_2", "role": "DEVELOPER" }
- Response: { "project_id": 1, "user_id": "usr_2", "role": "DEVELOPER" }
- Błędy: 403 ONLY_OWNER_ALLOWED, 409 USER_ALREADY_IN_PROJECT, 422

Lista członków

- Endpoint: GET /api/project/{project_id}/members
- Request: brak
- Response: [{ "project_id": 1, "user_id": "usr_2", "role": "DEVELOPER" }]
- Błędy: 403 NOT_A_PROJECT_MEMBER

Zmiana roli

- Endpoint: PATCH /api/project/{project_id}/members/{user_id}/role
- Request: { "role": "REVIEWER" }
- Response: { "project_id": 1, "user_id": "usr_2", "role": "REVIEWER" }
- Błędy: 403 ONLY_OWNER_ALLOWED, 403 CANNOT_CHANGE_OWNER_ROLE, 404 MEMBER_NOT_FOUND

Usunięcie członka

- Endpoint: DELETE /api/project/{project_id}/members/{user_id}
- Request: brak
- Response: 204 No Content

- Błędy: 403 ONLY_OWNER_ALLOWED, 403 CANNOT_REMOVE_PROJECT_OWNER, 404 MEMBER_NOT_FOUND

Kontrakty API – Integracje projektu

Lista integracji

- Endpoint: GET /api/project/{project_id}/integrations
- Request: brak
- Response: [{ "integration_id": 1, "project_id": 1, "provider": "GITHUB", "external_id": "owner/repo", "is_active": true }]

Podłączenie integracji

- Endpoint: POST /api/project/{project_id}/integrations
- Request: { "provider": "GITHUB", "external_id": "owner/repo", "access_token": "token" }
- Response: { "integration_id": 1, "project_id": 1, "provider": "GITHUB", "external_id": "owner/repo", "is_active": true }

Odłączenie integracji

- Endpoint: DELETE /api/project/{project_id}/integrations/{provider}
- Request: brak
- Response: 204 No Content

Walidacja i obsługa stanów UI

- Nieprawidłowy projectId w adresie URL skutkuje komunikatem „Nieprawidłowy identyfikator projektu”.
- Puste pole userId blokuje wysłanie formularza dodawania członka.
- Po dodaniu członka lista jest odświeżana, a formularz wraca do wartości domyślnych.
- Zmiana roli i usunięcie członka odświeżają cache/listę członków.
- Przycisk usunięcia jest zablokowany dla członka z rolą OWNER.
- Dla integracji wymagane są pola external_id i access_token.
- Stany ładowania i błędów są jawnie widoczne: „Wczytywanie...”, „Odświeżanie...”, komunikat błędu albo pustej listy.

- Błędy API powinny być tłumaczone na zrozumiałe komunikaty, np.
USER_ALREADY_IN_PROJECT → „Użytkownik jest już członkiem projektu”.

Uprawnienia w interfejsie

Backend pozostaje źródłem prawdy dla kontroli dostępu, natomiast frontend powinien ukrywać albo blokować akcje niedostępne dla danej roli. Rola OWNER może zarządzać projektem, członkami i integracjami. DEVELOPER oraz REVIEWER mogą przeglądać dane projektu zgodnie z przyznanym dostępem.

Funkcja UI	OWNER	DEVELOPER	REVIEWER
Wejście do szczegółów projektu	TAK	TAK	TAK
Lista członków projektu	TAK	TAK	TAK
Dodanie członka	TAK	NIE	NIE
Zmiana roli członka	TAK	NIE	NIE
Usunięcie członka	TAK, poza OWNERem	NIE	NIE
Podłączenie/odłączenie integracji	TAK	NIE	NIE
Podgląd integracji	TAK	TAK	TAK

Uwagi integracyjne do ujednolicenia

- Backend zwraca część pól jako snake_case, np. project_id i user_id, a frontend używa camelCase, np. projectId i userId. Z tego powodu productService mapuje odpowiedź API na modele TypeScript.
- Backendowy router projektu korzysta z prefiksu /api/project, a frontendowe apiFetch może automatycznie dodawać /api. W dokumentacji należy konsekwentnie rozróżniać ścieżkę backendową i ścieżkę wywoływaną z frontendu.
- Lista projektów w aktualnym froncie używa /projects, natomiast backendowy kontrakt projektu wskazuje /api/project/my. Finalnie należy zachować alias albo zmienić wywołanie frontendu na /project/my.
- Dokumentacja błędów powinna podawać kod HTTP i pole detail zwracane przez backend, ponieważ frontend może mapować je na komunikaty użytkownika.

Przykład implementacji – dodanie członka projektu

Uproszczony przepływ z widoku MembersPage: formularz blokuje pusty identyfikator użytkownika, a po udanej mutacji odświeża listę członków projektu.

```
const addMember = useMutation({
  mutationFn: () => {
    projectService.addMember(projectId, {
      userId: userId.trim(),
      role,
    }),
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['projects', 'members', projectId] })
    setUserId('')
    setRole('DEVELOPER')
  },
})
```

Przykład implementacji – podłączenie integracji

Formularz integracji wymaga wyboru dostawcy, identyfikatora zewnętrznego oraz tokena. Po sukcesie lista integracji jest pobierana ponownie, a pola formularza są czyszczone.

```
await integrationService.connect(projectId, {
  provider: 'GITHUB',
  external_id: 'owner/repo',
  access_token: 'token'
})
```

Kryteria akceptacji dla części Project & Membership

- Użytkownik po zalogowaniu widzi listę swoich projektów albo czytelny komunikat o braku projektów.
- Kliknięcie projektu prowadzi do szczegółów i umożliwia przejście do członków oraz integracji.
- Widok członków poprawnie pokazuje user_id oraz rolę każdego członka.
- Dodanie członka wysyła user_id i rolę do endpointu membership oraz po sukcesie odświeża tabelę.
- Zmiana roli wysyła PATCH z nową rolą i aktualizuje widok bez ręcznego odświeżania strony.
- Usunięcie członka wysyła DELETE, ale interfejs nie pozwala usunąć właściciela projektu.
- Widok integracji pozwala podłączyć GitHub/Jira/Confluence, pokazać aktywne integracje i odłączyć wybranego providera po potwierdzeniu.

- Wszystkie formularze mają obsługę ładowania, blokowanie niepoprawnego wysłania i komunikaty błędów.

User Service

Cel i zakres odpowiedzialności

Serwis "userService" (@/services/userService) odpowiada za obsługę operacji związanych z użytkownikami systemu. Zapewnia funkcjonalności pobierania listy wszystkich użytkowników, odczytu danych profilowych aktualnie zalogowanej osoby oraz modyfikacji jej danych osobowych i statusu konta.

Serwis ściśle współpracuje z mechanizmem lokalnej pamięci przeglądarki (localStorage) w celu identyfikacji zalogowanego użytkownika oraz ze stanem autoryzacji (np. "useAuthStore") w celu obsługi wylogowania i czyszczenia sesji.

Struktura danych (modele domenowe)

Model danych opiera się na interfejsach TypeScript zdefiniowanych w pliku "@/types/domain".

A. Typ "UserStatus"

Reprezentuje stany, w jakich może znajdować się konto użytkownika w bazie danych:

- ACTIVE (Konto aktywne - domyślny status użytkownika biorącego czynny udział w projektach)
- INACTIVE (Konto nieaktywne)
- BANNED (Konto zablokowane)

B. Interfejs "User"

Przedstawia pełną strukturę danych użytkownika przesyłaną przez API:

- userId (string): Unikalny identyfikator użytkownika.
- email (string): Adres e-mail.
- name (string): Imię użytkownika.
- surname (string): Nazwisko użytkownika.
- status (UserStatus): Status konta (ACTIVE, INACTIVE lub BANNED).
- isOnline (boolean): Flaga określająca, czy użytkownik jest aktualnie połączony z aplikacją.

Specyfikacja metod serwisu

1. listUsers()

- Opis: Pobiera pełną listę wszystkich zarejestrowanych użytkowników systemu.
- Parametry: Brak.
- Zwraca: Promise<User[]>

2. `getCurrentUser()`

- Opis: Pobiera szczegółowe informacje o profilu aktualnie zalogowanego użytkownika.
- Parametry: Brak (identyfikator użytkownika jest automatycznie pobierany z pamięci lokalnej przeglądarki pod kluczem 'userId').
- Wyjątki: Rzuca błąd (Error: 'Brak zalogowanego użytkownika'), jeśli w pamięci localStorage nie znaleziono klucza 'userId'.
- Zwraca: `Promise<User>`

3. `updateCurrentUser(data)`

- Opis: Aktualizuje dane profilowe oraz status konta zalogowanego użytkownika.
- Parametry:
 - `data` (obiekt): Zawiera pola "name" (string), "surname" (string), "status" (UserStatus) oraz "email" (string).
- Wyjątki: Rzuca błąd (Error: 'Brak zalogowanego użytkownika'), jeśli w localStorage nie znaleziono klucza 'userId'.
- Przebieg operacji: Metoda wykonuje kolejno dwa zapytania HTTP PUT: pierwsze aktualizuje podstawowe dane profilowe, drugie aktualizuje status konta użytkownika.
- Zwraca: `Promise<User>` (obiekt zaktualizowanego użytkownika zwrócony z drugiego zapytania).

Task Service

Cel i zakres odpowiedzialności

Serwis "taskService" (@/services/taskService) odpowiada za komunikację z API w zakresie podstawowego zarządzania zadaniami w projektach. Do jego głównych zadań należy pobieranie list zadań, szczegółów konkretnego zadania, tworzenie nowych zadań oraz aktualizacja statusów i przypisań użytkowników.

UWAGA DOTYCZĄCA PODZIAŁU ODPOWIEDZIALNOŚCI:

Aby zachować czytelność kodu i zasadę pojedynczej odpowiedzialności (Single Responsibility Principle), "taskService" NIE obsługuje dodatkowych aktywności powiązanych z zadaniem. Te operacje zostały wydzielone do osobnych serwisów:

- Komunikacja na czacie zadania, historia zmian oraz subskrypcje powiadomień: "taskActivityService"
- Zarządzanie dokumentacją zadania (dodawanie/edycja linków): "docsService"
- Tworzenie i weryfikacja Pull Requestów: "pullRequestService" lub "taskActivityService"

Struktura danych (modele domenowe)

A. Typ "TaskStatus"

Określa dopuszczalne stany, w jakich może znajdować się zadanie w systemie:

- TODO (Zadanie do wykonania)
- IN_PROGRESS (Zadanie w trakcie realizacji)
- IN_REVIEW (Zadanie w trakcie weryfikacji / kodu do przeglądu)
- DONE (Zadanie ukończone)
- CLOSED (Zadanie zamknięte)

B. Interfejs "Task"

Reprezentuje pełny obiekt zadania zwracany przez API:

- taskId (number): Unikalny identyfikator zadania.
- projectId (number): Identyfikator projektu, do którego należy zadanie.
- title (string): Tytuł zadania.
- description (string): Szczegółowy opis zadania.
- status (TaskStatus): Aktualny stan zadania.
- jiraIssueKey (string | null): Klucz powiązanego zadania w systemie Jira (np. "BON-107").
- gitBranchName (string | null): Nazwa powiązanej gałęzi w repozytorium Git.
- assigneeId (string | null): Identyfikator przypisanego programisty (wykonawcy).
- reviewerId (string | null): Identyfikator przypisanego recenzenta (osoby wykonującej code review).

Specyfikacja metod serwisu

Poniżej znajduje się opis wszystkich metod dostępnych w obiekcie "taskService":

1. `getProjectTasks(projectId, filters)`
 - Opis: Pobiera listę zadań powiązanych z wybranym projektem.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `filters (obiekt, opcjonalny)`: Pozwala filtrować zadania po statusie, wykonawcy (`assigneeld`), recenzencie (`reviewerId`) lub tylko obserwowanych (`onlySubscribed`).
 - Zwraca: `Promise<Task[]>`

2. `getTask(projectId, taskId)`
 - Opis: Pobiera szczegółowe dane pojedynczego zadania.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskId (number)`: Identyfikator zadania.
 - Zwraca: `Promise<Task>`

3. `getUserTasks(projectId, userId)`
 - Opis: Pobiera zadania przypisane do konkretnego użytkownika w danym projekcie.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `userId (string)`: Identyfikator użytkownika.
 - Zwraca: `Promise<Task[]>`

4. `getMyTasks(projectId)`
 - Opis: Szybkie pobranie zadań przypisanych do zalogowanego użytkownika (metoda uproszczona).
 - Zwraca: `Promise<Task[]>`

5. `createTask(projectId, taskData)`
 - Opis: Tworzy nowe zadanie w wybranym projekcie.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskData (obiekt)`: Zawiera pola "title" (string), "description" (string) oraz opcjonalnie "assigneeld" i "reviewerId".
 - Zwraca: `Promise<Task>`
 - Uwaga: Pola takie jak status, klucz Jira czy daty modyfikacji są automatycznie przypisywane przez serwer po odebraniu żądania.

6. `updateStatus(projectId, taskId, status)`
 - Opis: Zmienia status zadania (np. przejście z TODO do IN_PROGRESS).
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskId (number)`: Identyfikator zadania.
 - `status (TaskStatus)`: Nowy status.
 - Zwraca: `Promise<Task>`

7. `assignTask(projectId, taskId, assigneeId)`
 - Opis: Przypisuje wykonawcę do zadania lub usuwa przypisanie.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskId (number)`: Identyfikator zadania.
 - `assigneeId (string | null)`: Identyfikator użytkownika lub null w celu odpięcia osoby.
 - Zwraca: `Promise<Task>`

8. `assignReviewer(projectId, taskId, reviewerId)`
 - Opis: Przypisuje recenzenta do zadania lub usuwa przypisanie.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskId (number)`: Identyfikator zadania.
 - `reviewerId (string | null)`: Identyfikator użytkownika lub null w celu odpięcia osoby.
 - Zwraca: `Promise<Task>`

9. `requestClose(projectId, taskId)`
 - Opis: Wysyła żądanie zamknięcia zadania do backendu.
 - Parametry:
 - `projectId (number)`: Identyfikator projektu.
 - `taskId (number)`: Identyfikator zadania.
 - Zwraca: `Promise<void>`

Przykład implementacji w komponencie React

Poniższy uproszczony przykład obrazuje prawidłowy sposób pobierania danych oraz modyfikacji przypisania zadania z poziomu komponentu React:

```

import { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { taskService } from '@services/taskService';
import type { Task } from '@types/domain';

export default function TaskManager() {
  const { projectId, taskId } = useParams<{ projectId: string; taskId: string }>();
  const [task, setTask] = useState<Task | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (!projectId || !taskId) return;

    setLoading(true);
    taskService.getTask(Number(projectId), Number(taskId))
      .then((data) => {
        setTask(data);
        setError(null);
      })
      .catch((err) => {
        console.error(err);
        setError("Wystąpił błąd podczas pobierania szczegółów zadania.");
      })
      .finally(() => {
        setLoading(false);
      });
  }, [projectId, taskId]);

  const handleStatusUpdate = async (newStatus: any) => {
    if (!projectId || !taskId || !task) return;
    try {
      const updatedTask = await taskService.updateStatus(Number(projectId), Number(taskId), newStatus);
      setTask(updatedTask);
    } catch (err) {
      console.error("Nie udało się zaktualizować statusu", err);
      alert("Wystąpił błąd podczas próby aktualizacji statusu.");
    }
  };

  if (loading) return <div>Ładowanie danych zadania...</div>;
  if (error) return <div style={{ color: 'red' }}>{error}</div>;
  if (!task) return <div>Nie znaleziono zadania.</div>;

  return (
    <div>
      <h1>{task.title}</h1>

```

```
<p>{task.description}</p>
<p>Aktualny status: {task.status}</p>
<button onClick={() => handleStatusUpdate('IN_PROGRESS')}>
  Ustaw status: W trakcie realizacji
</button>
</div>
);
}
```

Docs Service

Cel i zakres odpowiedzialności

Docs Service odpowiada za zarządzanie dokumentacją powiązaną z zadaniami projektu. Moduł umożliwia tworzenie, pobieranie oraz aktualizację dokumentacji przypisanej do konkretnego zadania. Dokumentacja stanowi obowiązkowy element procesu review i jest wykorzystywana przez moduł Pull Request podczas tworzenia zgłoszenia do weryfikacji.

Serwis odpowiada również za walidację poprawności operacji związanych z dokumentacją, w szczególności za kontrolę istnienia zadania oraz zapobieganie tworzeniu wielu wpisów dokumentacji dla tego samego zadania.

Struktura danych

Model dokumentacji zawiera następujące pola:

- docsid – identyfikator dokumentacji,
- taskid – identyfikator zadania,
- title – tytuł dokumentacji,
- url – adres URL dokumentacji,
- externalid – identyfikator zewnętrzny dokumentacji.

Do tworzenia i aktualizacji dokumentacji wykorzystywane są pola:

- title,
- url.

Zasady walidacji i ograniczenia

- Dokumentacja może zostać utworzona wyłącznie dla istniejącego zadania.
- Dla jednego zadania może istnieć tylko jeden wpis dokumentacji.
- Próba utworzenia kolejnej dokumentacji dla tego samego zadania kończy się błędem walidacji.
- Aktualizacja dokumentacji jest możliwa wyłącznie dla istniejącego wpisu.
- Dokumentacja jest wymagana przed utworzeniem Pull Requesta.

Specyfikacja metod serwisu

addDocs(projectId, taskId, request)

Tworzy nowy wpis dokumentacji dla wskazanego zadania.

W trakcie operacji wykonywane są następujące kroki:

1. Weryfikacja istnienia zadania.
2. Sprawdzenie, czy dokumentacja dla zadania nie została już utworzona.

3. Wygenerowanie identyfikatora externalId.
4. Zapis danych w repozytorium.

getTaskDocs(projectId, taskId)

Pobiera dokumentację przypisaną do wskazanego zadania.

Metoda sprawdza istnienie zadania oraz obecność dokumentacji.

updateTaskDocs(projectId, taskId, request)

Aktualizuje dane istniejącej dokumentacji.

Możliwa jest modyfikacja tytułu oraz adresu URL dokumentacji.

getProjectDocs(projectId)

Pobiera wszystkie dokumentacje powiązane z zadaniami należącymi do wskazanego projektu.

requireTaskHasDocs(taskId)

Sprawdza, czy wskazane zadanie posiada dokumentację.

Metoda jest wykorzystywana przez PR Service przed utworzeniem Pull Requesta.

Jeżeli dokumentacja nie istnieje, zwracany jest błąd TASK_DOCS_REQUIRED.

Endpointy API

POST /api/projects/{projectId}/tasks/{taskId}/docs

Opis:

Tworzy dokumentację dla wskazanego zadania.

Parametry ścieżki:

- projectId – identyfikator projektu,
- taskId – identyfikator zadania.

Request Body:

```
{  
  "title": "string",  
  "url": "string"  
}
```

Odpowiedź:

201 Created

```
{
```



```
"docslId": 1,  
"taskId": 1,  
"title": "string",  
"url": "string",  
"externalId": "external-docs-url-1-1"  
}
```

Błędy:

- 404 TASK_NOT_FOUND – zadanie nie istnieje,
- 409 DOCS_ALREADY_EXISTS – dokumentacja dla tego zadania już istnieje,
- 422 Unprocessable Entity – niepoprawne dane wejściowe, np. pusty title lub url.

GET /api/projects/{projectId}/tasks/{taskId}/docs**Opis:**

Pobiera dokumentację przypisaną do wskazanego zadania.

Parametry ścieżki:

- projectId – identyfikator projektu,
- taskId – identyfikator zadania.

Odpowiedź:

200 OK

```
{  
  "docslId": 1,  
  "taskId": 1,  
  "title": "string",  
  "url": "string",  
  "externalId": "external-docs-url-1-1"  
}
```

Błędy:

- 404 TASK_NOT_FOUND – zadanie nie istnieje,
- 404 DOCS_NOT_FOUND – dokumentacja dla zadania nie istnieje.

PUT /api/projects/{projectId}/tasks/{taskId}/docs**Opis:**

Aktualizuje dokumentację przypisaną do wskazanego zadania.

Parametry ścieżki:

- projectId – identyfikator projektu,
- taskId – identyfikator zadania.

Request Body:

```
{
  "title": "string",
  "url": "string"
}
```

Odpowiedź:

200 OK

```
{
  "docslid": 1,
  "taskld": 1,
  "title": "string",
  "url": "string",
  "externalld": "external-docs-url-1-1"
}
```

Błędy:

- 404 TASK_NOT_FOUND – zadanie nie istnieje,
- 404 DOCS_NOT_FOUND – dokumentacja dla zadania nie istnieje,
- 422 Unprocessable Entity – niepoprawne dane wejściowe, np. pusty title lub url.

GET /api/projects/{projectId}/docs**Opis:**

Pobiera listę dokumentacji powiązanych z zadaniami należącymi do wskazanego projektu.

Parametry ścieżki:

- projectId – identyfikator projektu.

Odpowiedź:

200 OK

```
[
  {
    "docslid": 1,
    "taskld": 1,
    "title": "string",
    "url": "string",
    "externalld": "external-docs-url-1-1"
  }
]
```

Błędy:

- 404 PROJECT_NOT_FOUND – projekt nie istnieje.

PR Service

Cel i zakres odpowiedzialności

PR Service odpowiada za obsługę procesu review zmian realizowanych w ramach zadania. Moduł umożliwia tworzenie Pull Requestów, pobieranie ich szczegółów, filtrowanie list zgłoszeń oraz obsługę procesu akceptacji lub odrzucenia zmian.

Serwis współpracuje z modułami Task Service oraz Docs Service, zapewniając poprawny przepływ zadania przez etapy review.

Struktura danych

Model Pull Request zawiera następujące pola:

- pullRequestId,
- projectId,
- taskId,
- externalId,
- title,
- url,
- assigneeId,
- reviewerId,
- status,
- createdAt,
- updatedAt,
- mergedAt.

Zasady walidacji i ograniczenia

- Pull Request może zostać utworzony wyłącznie dla istniejącego zadania.
- Zadanie musi posiadać dokumentację.
- Dla jednego zadania może istnieć tylko jeden Pull Request.
- Utworzenie Pull Requesta powoduje zmianę statusu zadania na IN_REVIEW.
- Odrzucenie Pull Requesta wymaga podania powodu odrzucenia.

Specyfikacja metod serwisu

createPullRequest(projectId, taskId)

Tworzy Pull Request dla wskazanego zadania.

W trakcie wykonywania metody:

1. Weryfikowane jest istnienie zadania.
2. Sprawdzana jest obecność dokumentacji zadania.
3. Sprawdzane jest, czy Pull Request nie został już utworzony.
4. Generowany jest tytuł Pull Requesta.
5. Tworzony jest wpis Pull Request.
6. Aktualizowany jest status zadania na IN_REVIEW.

getPullRequest(projectId, pullRequestId)

Pobiera szczegóły wskazanego Pull Requesta.

getProjectPullRequests(projectId, filters)

Pobiera listę Pull Requestów należących do projektu wraz z możliwością filtrowania po statusie oraz reviewerze.

getUserPullRequests(userId)

Pobiera Pull Requesty, w których użytkownik jest reviewerem albo assignee zadania. Filtrowanie po statusie wykonywane jest na poziomie endpointu API.

acceptReview(projectId, pullRequestId)

Akceptuje Pull Request. Metoda sprawdza, czy Pull Request ma status OPEN, a następnie zmienia jego status na MERGED, ustawia mergedAt oraz zmienia status powiązanego zadania na DONE.

rejectReview(projectId, pullRequestId, reason)

Odrzuca Pull Request wraz z przekazanym powodem odrzucenia. Metoda sprawdza, czy Pull Request ma status OPEN, a następnie zmienia jego status na CLOSED oraz ustawia status powiązanego zadania na IN_PROGRESS.

getReviewsScopePullRequests(userId, status)

Pobiera Pull Requesty widoczne w trybie reviews.

Endpointy API

POST /api/projects/{projectId}/tasks/{taskId}/pull-requests

Opis:

Tworzy Pull Request dla wskazanego zadania.

Parametry ścieżki:

- projectId – identyfikator projektu,
- taskId – identyfikator zadania.

Request Body:

Brak.

Odpowiedź:

201 Created

```
{
  "pullRequestId": 1,
  "projectId": null,
  "taskId": 1,
  "externalId": "string",
  "title": "PR for task 1: Task title",
  "url": "string",
  "assigneeId": null,
  "reviewerId": "string",
  "status": "OPEN",
  "createdAt": "2026-01-01T12:00:00",
  "updatedAt": "2026-01-01T12:00:00",
  "mergedAt": null
}
```

Błędy:

- 404 TASK_NOT_FOUND – zadanie nie istnieje,
- 409 TASK_DOCS_REQUIRED – zadanie nie posiada wymaganej dokumentacji,
- 409 PULL_REQUEST_ALREADY_EXISTS – Pull Request dla tego zadania już istnieje.

GET /api/projects/{projectId}/pull-requests/{pullRequestId}**Opis:**

Pobiera szczegóły wskazanego Pull Requesta.

Parametry ścieżki:

- projectId – identyfikator projektu,
- pullRequestId – identyfikator Pull Requesta.

Odpowiedź:

200 OK

```
{
  "pullRequestId": 1,
  "projectId": null,
  "taskId": 1,
  "externalId": "string",
  "title": "PR for task 1: Task title",
  "url": "string",
}
```

```
"assigneeId": null,  
"reviewerId": "string",  
"status": "OPEN",  
"createdAt": "2026-01-01T12:00:00",  
"updatedAt": "2026-01-01T12:00:00",  
"mergedAt": null  
}
```

Błędy:

- 404 PULL_REQUEST_NOT_FOUND – Pull Request nie istnieje w podanym projekcie.

GET /api/projects/{projectId}/pull-requests

Opis:

Pobiera listę Pull Requestów należących do wskazanego projektu.

Parametry ścieżki:

- projectId – identyfikator projektu.

Parametry zapytania:

- reviewerId – opcjonalny identyfikator reviewera,
- status – opcjonalny status Pull Requesta; możliwe wartości: OPEN, MERGED, CLOSED.

Odpowiedź:

200 OK

```
[  
  {  
    "pullRequestId": 1,  
    "projectId": null,  
    "taskId": 1,  
    "externalId": "string",  
    "title": "PR for task 1: Task title",  
    "url": "string",  
    "assigneeId": null,  
    "reviewerId": "string",  
    "status": "OPEN",  
    "createdAt": "2026-01-01T12:00:00",  
    "updatedAt": "2026-01-01T12:00:00",  
    "mergedAt": null  
  }  
]
```

GET /api/users/{userId}/pull-requests

Opis:

Pobiera Pull Requesty powiązane z użytkownikiem. Domyślnie zwracane są Pull Requesty, w których użytkownik jest reviewerem albo wykonawcą zadania. Dla parametru mode=reviews stosowany jest osobny tryb pobierania Pull Requestów do review.

Parametry ścieżki:

- userId – identyfikator użytkownika.

Parametry zapytania:

- status – opcjonalny status Pull Requesta; możliwe wartości: OPEN, MERGED, CLOSED.
- mode – opcjonalny tryb pobierania; wartość reviews uruchamia widok zgłoszeń do review.

Odpowiedź:

200 OK

```
[
  {
    "pullRequestId": 1,
    "projectId": 1,
    "taskId": 1,
    "externalId": "string",
    "title": "PR for task 1: Task title",
    "url": "string",
    "assigneeId": "string",
    "reviewerId": "string",
    "status": "OPEN",
    "createdAt": "2026-01-01T12:00:00",
    "updatedAt": "2026-01-01T12:00:00",
    "mergedAt": null
  }
]
```

POST /api/projects/{projectId}/pull-requests/{pullRequestId}/accept

Opis:

Akceptuje Pull Request. Po akceptacji status Pull Requesta zostaje zmieniony na MERGED, ustawiana jest data mergedAt, a powiązane zadanie przechodzi do statusu DONE.

Parametry ścieżki:

- projectId – identyfikator projektu,
- pullRequestId – identyfikator Pull Requesta.

Request Body:

Brak.

Odpowiedź:

200 OK

```
{
  "pullRequestId": 1,
  "projectId": null,
  "taskId": 1,
  "externalId": "string",
  "title": "PR for task 1: Task title",
  "url": "string",
  "assigneeId": null,
  "reviewerId": "string",
  "status": "MERGED",
  "createdAt": "2026-01-01T12:00:00",
  "updatedAt": "2026-01-01T12:10:00",
  "mergedAt": "2026-01-01T12:10:00"
}
```

Błędy:

- 404 PULL_REQUEST_NOT_FOUND – Pull Request nie istnieje w podanym projekcie,
- 409 PULL_REQUEST_NOT_OPEN – Pull Request nie ma statusu OPEN.

POST /api/projects/{projectId}/pull-requests/{pullRequestId}/reject**Opis:**

Odrzuca Pull Request z podanym powodem odrzucenia. Po odrzuceniu status Pull Requesta zostaje zmieniony na CLOSED, a powiązane zadanie wraca do statusu IN_PROGRESS.

Parametry ścieżki:

- projectId – identyfikator projektu,
- pullRequestId – identyfikator Pull Requesta.

Request Body:

```
{
  "reason": "string"
}
```


Odpowiedź:

200 OK

```
{
  "pullRequestId": 1,
  "projectId": null,
  "taskId": 1,
  "externalId": "string",
  "title": "PR for task 1: Task title",
  "url": "string",
  "assigneeId": null,
  "reviewerId": "string",
  "status": "CLOSED",
  "createdAt": "2026-01-01T12:00:00",
  "updatedAt": "2026-01-01T12:10:00",
  "mergedAt": null
}
```

Błędy:

- 404 PULL_REQUEST_NOT_FOUND – Pull Request nie istnieje w podanym projekcie,
- 409 PULL_REQUEST_NOT_OPEN – Pull Request nie ma statusu OPEN,
- 422 Unprocessable Entity – niepoprawne dane wejściowe, np. pusty reason.

Chat service

Cel i zakres odpowiedzialności

Serwis **ChatService** odpowiada za obsługę komunikacji tekstowej w ramach zadań projektowych. Umożliwia pobieranie historii wiadomości, wysyłanie nowych wiadomości, edycję i usuwanie istniejących wiadomości oraz synchronizację zmian w czasie rzeczywistym za pomocą WebSocket.

Serwis współpracuje z:

- ChatRepository – operacje na bazie danych,
- modelem ChatMessage,
- modelem User – pobieranie adresów e-mail autorów,
- chat_manager – rozgłaszanie zdarzeń WebSocket.

Struktura danych

ChatMessageResponse

- messageId (int) – identyfikator wiadomości,
- taskId (int) – identyfikator zadania,
- authorId (string) – identyfikator autora,
- authorEmail (string | null) – adres e-mail autora,
- text (string) – treść wiadomości,
- createdAt (datetime) – data utworzenia.

SendMessageRequest / UpdateMessageRequest

- text (string) – treść wiadomości.

Specyfikacja metod serwisu

get_messages(task_id)

Pobiera wszystkie wiadomości przypisane do zadania wraz z adresami e-mail autorów.

Parametry: task_id (int)

Zwraca: list[ChatMessageResponse]

send_message(task_id, author_id, text)

Tworzy nową wiadomość oraz wysyła zdarzenie WebSocket typu created.

Parametry: task_id (int), author_id (string), text (string)

Zwraca: ChatMessageResponse

update_message(message_id, current_user_id, text)

Aktualizuje treść wiadomości. Operacja dostępna wyłącznie dla autora wiadomości.

Parametry: message_id (int), current_user_id (string), text (string)

Wyjątki: 404 Message not found, 403 Forbidden

Zwraca: ChatMessageResponse

delete_message(message_id, current_user_id)

Usuwa wiadomość. Operacja dostępna wyłącznie dla autora wiadomości.

Parametry: message_id (int), current_user_id (string)

Wyjątki: 404 Message not found, 403 Forbidden

Endpointy API

GET /projects/{project_id}/tasks/{task_id}/messages

Pobiera historię wiadomości zadania.

Zwraca: list[ChatMessageResponse]

POST /projects/{project_id}/tasks/{task_id}/messages

Tworzy nową wiadomość.

Body:

```
{  
  "text": "Treść wiadomości"  
}
```

Autoryzacja: wymagana

PATCH /projects/{project_id}/tasks/{task_id}/messages/{message_id}

Aktualizuje treść wiadomości.

Autoryzacja: wymagana (autor wiadomości)

DELETE /projects/{project_id}/tasks/{task_id}/messages/{message_id}

Usuwa wiadomość.

Autoryzacja: wymagana (autor wiadomości)

Zwraca: 204 No Content

Komunikacja websocket

Endpoint:

/ws/tasks/{task_id}/chat

Umożliwia nasłuchiwanie zmian na czacie danego zadania w czasie rzeczywistym.

Obsługiwane zdarzenia:

```
{  
  "type": "created",  
  "message": {...}  
}  
  
{  
  "type": "updated",  
  "message": {...}  
}  
  
{  
  "type": "deleted",  
  "messageId": 15  
}
```

Dzięki komunikacji WebSocket wszystkie zmiany są automatycznie synchronizowane pomiędzy użytkownikami bez konieczności odświeżania danych.

Analytics Service

Cel i zakres odpowiedzialności

Serwis **AnalyticsService** odpowiada za generowanie statystyk i wskaźników dotyczących realizacji zadań w projekcie. Dostarcza dane wykorzystywane do monitorowania postępu prac, obciążenia użytkowników oraz efektywności realizacji zadań.

Serwis współpracuje z:

- AnalyticsRepository – pobieranie danych statystycznych z bazy,
- modelami Task i User,
- endpointem analitycznym projektu.

Struktura danych

AnalyticsOverviewResponse

Podstawowe wskaźniki:

- taskCount (int) – liczba wszystkich zadań,
- doneTasks (int) – liczba ukończonych zadań,
- openTasks (int) – liczba nieukończonych zadań,
- completionRate (float) – procent ukończonych zadań,
- unassignedTasks (int) – liczba nieprzypisanych zadań,
- staleTasks (int) – liczba zadań nieaktualizowanych od co najmniej 7 dni,
- avgCycleTimeHours (float | null) – średni czas realizacji zadania w godzinach.

Zestawienia:

- tasksByStatus (Dict[str, int]) – liczba zadań według statusów,
- tasksByUser (Dict[str, int]) – liczba zadań przypisanych użytkownikom,
- tasksByReviewer (Dict[str, int]) – liczba zadań przypisanych recenzentom,
- wipByUser (Dict[str, int]) – liczba aktywnych zadań użytkowników (IN_PROGRESS, IN_REVIEW),
- throughputByDay (Dict[str, int]) – liczba ukończonych zadań w poszczególnych dniach.

Specyfikacja metod serwisu

get_overview(project_id)

Opis: Generuje pełne podsumowanie statystyk projektu.

Parametry: project_id (int) – identyfikator projektu.

Przebieg operacji:

1. Pobranie liczby wszystkich zadań.
2. Pobranie liczby zadań ukończonych, nieprzypisanych oraz nieaktualizowanych.
3. Grupowanie zadań według statusów, wykonawców i recenzentów.
4. Obliczenie wskaźnika ukończenia projektu.

5. Wyznaczenie średniego czasu realizacji zadań.
6. Wyliczenie dziennej przepustowości (throughput).
7. Zwrot kompletnego zestawu statystyk.

Zwraca: AnalyticsOverviewResponse

Endpointy API

GET /projects/{project_id}/analytics/overview

Opis: Pobiera pełny zestaw statystyk i wskaźników dla wskazanego projektu.

Zwraca: AnalyticsOverviewRespons

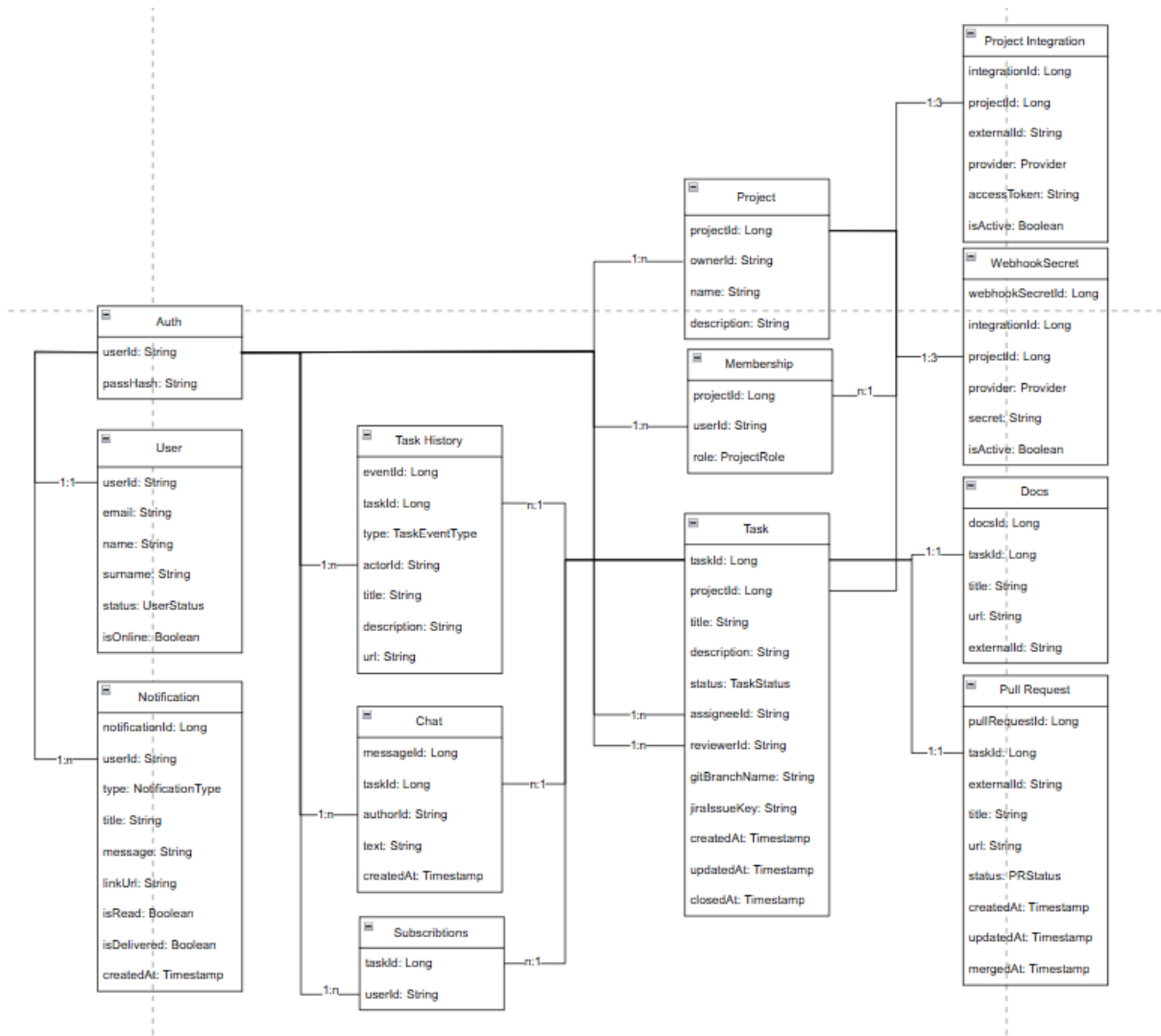
DOSTĘPNE METRYKI

- Łączna liczba zadań.
- Liczba zadań ukończonych i otwartych.
- Wskaźnik ukończenia projektu (%).
- Liczba zadań bez przypisanego wykonawcy.
- Liczba zadań nieaktualizowanych przez ponad 7 dni.
- Średni czas realizacji zadania.
- Rozkład zadań według statusów.
- Rozkład zadań według wykonawców.
- Rozkład zadań według recenzentów.
- Liczba aktywnych zadań przypadających na użytkownika (WIP).
- Liczba ukończonych zadań w kolejnych dniach (throughput).

Database structure

Struktura bazy danych została zaprojektowana wokół głównych encji systemu, takich jak projekty, zadania, użytkownicy oraz ich interakcje. Każda tabela odpowiada konkretnemu obszarowi funkcjonalnemu i wspiera realizację przypadków użycia opisanych w systemie.

- **Project** — przechowuje podstawowe informacje o projekcie, w tym właściciela oraz dane opisowe.
- **Membership** — łączy użytkowników z projektami oraz określa ich role i uprawnienia w danym projekcie.
- **User** — zawiera dane użytkownika oraz jego profil, status dostępności oraz informację o byciu online.
- **Auth** — przechowuje dane uwierzytelniające użytkownika, wykorzystywane podczas logowania.
- **Task** — centralna tabela systemu, przechowująca informacje o zadaniach, ich statusie, przypisaniach oraz powiązaniach z systemami zewnętrznymi (Jira, Git).
- **Task History** — zapisuje historię zdarzeń związanych z zadaniem, umożliwiając śledzenie jego przebiegu oraz synchronizacji z zewnętrznymi systemami.
- **Pull Request** — reprezentuje proces review, przechowując informacje o stanie pull requesta oraz przypisanym reviewrze.
- **Chat** — przechowuje wiadomości powiązane z zadaniami, umożliwiając komunikację zespołu w kontekście konkretnego zadania.
- **Notification** — zawiera powiadomienia użytkowników wraz z ich stanem (odczytane, dostarczone).
- **Subscriptions** — definiuje, którzy użytkownicy obserwują konkretne zadania i powinni otrzymywać powiadomienia o zmianach.
- **Project Integration** — przechowuje konfigurację integracji z zewnętrznymi systemami (np. Git, Jira), w tym dane dostępowe oraz identyfikatory zasobów.
- **Webhook Secret** — przechowuje sekrety wykorzystywane do weryfikacji webhooków z zewnętrznymi systemami, zapewniając bezpieczeństwo komunikacji.



Project

Tabela przechowuje podstawowe informacje o projekcie.

- ProjectId: Long — unikalny identyfikator projektu.
- OwnerId: String — identyfikator użytkownika, który jest właścicielem projektu
- Name: String — nazwa projektu.
- Description: String — opis projektu.

ProjectMember

Tabela przechowuje przynależność użytkowników do projektów.

- ProjectId: Long — identyfikator projektu.
- UserId: String — identyfikator użytkownika.
- Role: ProjectRole — rola użytkownika w projekcie.

User

Tabela przechowuje dane użytkownika.

- UserId: String — unikalny identyfikator użytkownika.
- Email: String — adres email użytkownika.
- Name: String — imię użytkownika.
- Surname: String — nazwisko użytkownika.
- Status: UserStatus — status dostępności użytkownika.
- IsOnline: Boolean — czy użytkownik jest aktualnie online.

Auth

Tabela przechowuje dane uwierzytelniające.

- UserId: String — identyfikator użytkownika.
- PassHash: String — zahashowane hasło.

ProjectIntegration

Tabela przechowuje integracje projektu.

- IntegrationId: Long — unikalny identyfikator integracji.
- ProjectId: Long — identyfikator projektu.
- ExternalId: String — identyfikator projektu w systemie zewnętrznym.
- Provider: IntegrationProvider — typ integracji (GitHub, Jira itd.).
- AccessToken: String — token dostępu.
- IsActive: Boolean — czy integracja jest aktywna.

Task

Tabela przechowuje zadania w projekcie.

- TaskId: Long — unikalny identyfikator zadania.
- ProjectId: Long — identyfikator projektu.
- Title: String — tytuł zadania.
- Description: String — opis zadania.
- Status: TaskStatus — status zadania.
- AssigneeId: String — użytkownik przypisany do zadania.
- ReviewerId: String — użytkownik odpowiedzialny za review.
- GitBranchName: String — nazwa gałęzi Git powiązanej z zadaniem.
- JiraIssueKey: String — identyfikator zadania w Jira.
- CreatedAt: LocalDateTime — data utworzenia.
- UpdatedAt: LocalDateTime — data aktualizacji.
- ClosedAt: LocalDateTime — data zamknięcia zadania.

TaskSubscription

Tabela przechowuje subskrypcje zadań.

- TaskId: Long — identyfikator zadania.
- UserId: String — użytkownik subskrybujący zadanie. **Docs**

Tabela przechowuje dokumentację powiązaną z zadaniem.

- DocId: Long — unikalny identyfikator dokumentu.
- TaskId: Long — identyfikator zadania.
- Title: String — tytuł dokumentacji.
- Url: String — link do dokumentacji.
- ExternalId: String — identyfikator w zewnętrznym systemie.

WebhookSecret

Tabela przechowuje sekrety webhooków.

- WebhookSecretId: Long — unikalny identyfikator sekretu.
- IntegrationId: Long — powiązana integracja.
- ProjectId: Long — identyfikator projektu.
- Provider: IntegrationProvider — dostawca webhooka.
- Secret: String — sekret używany do podpisu webhooków.
- IsActive: Boolean — czy sekret jest aktywny.

ChatMessage

Tabela przechowuje wiadomości w czacie zadania.

- MessageId: Long — unikalny identyfikator wiadomości.
- TaskId: Long — identyfikator zadania.
- AuthorId: String — autor wiadomości.
- Text: String — treść wiadomości.
- CreatedAt: LocalDateTime — data wysłania.

PullRequest

Tabela przechowuje pull requesty.

- PullRequestId: Long — unikalny identyfikator PR.
- TaskId: Long — identyfikator zadania.
- ExternalId: String — identyfikator w systemie zewnętrznym.
- Title: String — tytuł PR.
- Url: String — link do PR.
- ReviewerId: String — reviewer.
- Status: PullRequestStatus — status PR.
- CreatedAt: LocalDateTime — data utworzenia.
- UpdatedAt: LocalDateTime — data aktualizacji.
- MergedAt: LocalDateTime — data mergowania.

TaskHistory

Tabela przechowuje historię zdarzeń zadania.

- EventId: Long — unikalny identyfikator zdarzenia.
- TaskId: Long — identyfikator zadania.
- Type: TaskEventType — typ zdarzenia.
- ActorId: String — użytkownik wykonujący akcję.
- Title: String — tytuł zdarzenia.
- Description: String — opis zdarzenia.
- Url: String — link powiązany ze zdarzeniem.

Notification

Tabela przechowuje powiadomienia użytkowników.

- NotificationId: Long — unikalny identyfikator powiadomienia.
- UserId: String — odbiorca powiadomienia.
- Type: NotificationType — typ powiadomienia.
- Title: String — tytuł powiadomienia.
- Message: String — treść powiadomienia.
- LinkUrl: String — link powiązany z powiadomieniem.
- IsRead: Boolean — czy powiadomienie zostało przeczytane.
- IsDelivered: Boolean — czy zostało dostarczone online.
- CreatedAt: LocalDateTime — data utworzenia.